NOTES

HOW AN AGENTIC APP WORKS, IN PLAIN WORDS

Memorandum on Agentic AI Applications · 21 Oct 2026

Contents

I. The big picture
II. The one loop every agent uses
III. The full flow, step by step
IV. How the project is organised
V. All the types of agent apps, with examples
VI. Five rules every real agentic app follows
VII. In one line
VIII. Reading list

Every red underlined title is a clickable link to the paper or guide that explains that part in full.

I. THE BIG PICTURE

An agentic app has three parts working together:

1. The app. This is what the user sees and touches: website, phone app, chat window, or bot.
2. The server. It sits in the middle. It checks who the user is, keeps the secret keys safe, and calls the agent.
3. The agent. This is the "brain". It is an AI model plus instructions plus tools, and it runs in a loop until the job is done.

The user never talks to the AI model directly. They talk to the app, the app talks to the server, and the server runs the agent.

> READ THE PAPER
> ▸ Building effective agents (Anthropic, 2024): the clearest guide to what an agent is and the common app patterns.
> ▸ LLM Powered Autonomous Agents (Lilian Weng, 2023): model + planning + memory + tools, explained with diagrams.

II. THE ONE LOOP EVERY AGENT USES

Each time the agent gets a task, it repeats the same cycle:

1. Think. It reads the task and decides: "Can I answer now, or do I need more information?"
2. Use a tool. If it needs information, it picks a tool, such as "look up order", "search docs" or "run tests".
3. Read the result. The tool's answer goes back to the agent.
4. Repeat or finish. If it has enough, it writes the final answer. If not, it goes back to step 1.

A safety limit stops the loop after a set number of rounds, so it can never run forever.

Example: "Where is my order 123?"

- Think: "I don't know this. I need the order data."
- Tool: looks up order 123
- Result: "Shipped, arrives 22 Oct."
- Think: "Now I can answer."
- Final answer: "Your order has shipped and will arrive on 22 October."

> READ THE PAPER
> ▸ ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022): the paper behind the think, use a tool, read the result loop.
> ▸ Toolformer: Language Models Can Teach Themselves to Use Tools (Schick et al., 2023): how a model learns when and how to call a tool.

III. THE FULL FLOW, STEP BY STEP

Step 1: The user asks. The user types or speaks a request in the app.

Step 2: The app sends it to the server. The app does not think. It only passes the message on.

Step 3: The server checks the user. It confirms the user is logged in and notes who they are. This matters because the agent must only see this user's data.

Step 4: The server starts the agent. It gives the agent three things:

- the user's message
- the instructions (who it is, what it may and may not do, how to reply)
- the list of tools it may use for this user

Step 5: The agent runs the loop. It thinks, uses a tool, and reads the result, as many times as needed.

Step 6: The tools do the real work. Tools are the agent's hands. They read the database, call other services, and send messages. The agent only decides which

tool to use and what to ask it.

Step 7: The answer streams back. The words come back through the server to the app and appear on screen word by word, so the user isn't left waiting.

Step 8 (optional): Long jobs. If the task takes minutes or hours (a report, 100 emails), the server puts it in a queue. A background worker runs the agent, and the user gets a notice when it is done.

READ THE GUIDE

- Tool use with Claude (Anthropic docs): how the server hands tools to the model and runs them.
- Streaming messages (Anthropic docs): how the answer comes back word by word.
- Durable execution (LangGraph docs): long jobs that survive restarts and run in the background.

IV. HOW THE PROJECT IS ORGANISED

FOLDER	WHAT LIVES THERE	EXAMPLE
App / screens	What the user sees	chat page, dashboard, settings
Server routes	The middle layer	"receive chat message", "check login"
Agent folder	The brain	instructions, the list of tools, model choice
Tools folder	The agent's hands	"get order", "search docs", "book slot"
Database / shared	Data both sides use	users, orders, notes
Worker	Long background jobs	reports, scheduled tasks
Tests / evals	Checks the agent answers well	a list of sample questions and expected answers

Keeping the agent in its own folder means you can change the AI model or improve the instructions without touching the rest of the app.

READ THE GUIDE

- 12-Factor Agents (HumanLayer): rules for building agents that are real, testable software.
- A Practical Guide to Building Agents (OpenAI, PDF): model, tools, instructions, and guardrails as separate parts.
- Writing effective tools for agents (Anthropic): how to design the tools folder and test it with evals.

V. ALL THE TYPES OF AGENT APPS, WITH EXAMPLES

1. Single agent with tools (the basic type)

One agent, a few tools, and one loop.

- Flow: User asks, the agent looks something up, then answers.
- Examples:
  - FAQ bot on a website that searches the help pages
  - Order status bot that looks up the order
  - Repo Q&A bot: "Where is login handled?" It searches the code and answers.
  - Log reader that reads server logs and explains the errors

> READ THE PAPER
> ▸ ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022): the paper behind the think, use a tool, read the result loop.
> ▸ MRKL Systems (Karpas et al., 2022): early design of one model routing work to tools.

2. Router + specialist agents (handoff / multi-agent)

A small "front desk" agent decides who should handle the request, then hands it over.

- Flow: User asks, the triage agent picks the right specialist, the specialist uses its tools, then answers.
- Examples:
  - Support desk: triage sends the user to the billing, tech or refund agent
  - Travel planner: separate flight, weather and budget agents
  - Stack: an in-app help agent answers questions and passes bug reports to a bug-triage agent

> READ THE PAPER
> ▸ Handoffs (OpenAI Agents SDK docs): how a triage agent hands a user over to a specialist.
> ▸ How we built our multi-agent research system (Anthropic): a lead agent splitting work across sub-agents.

3. Chat with your documents (RAG)

The agent first finds the right pieces of your documents, then answers from them.

- Flow: User uploads files, the app breaks them into small pieces and stores them. When the user asks, the agent searches those pieces and answers from

what it found.

- Examples:
  - Chat with your PDFs
  - Company handbook assistant
  - "Careful RAG": if the found documents are poor, the agent rewrites the question and searches again

> READ THE PAPER
> ▸ Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020): the original RAG paper: find pieces first, then answer.
> ▸ Self-RAG (Asai et al., 2023): the model judges its own search results before answering.
> ▸ Corrective RAG (Yan et al., 2024): the "careful RAG" idea: poor results, so search again.

4. Structured output (fill a fixed form)

The agent doesn't chat. It returns data in a fixed shape the app can use directly.

- Flow: Input goes in, the agent reads and sorts it, and a clean structured result comes out to be saved or shown.
- Examples:
  - Meeting notes: transcript in, summary and task list out
  - Resume screener: CVs in, a score for each job out
  - Stack "Remember": your highlights in, quiz questions out

> READ THE PAPER
> ▸ Introducing Structured Outputs in the API (OpenAI): forcing the answer into a fixed JSON shape.
> ▸ Efficient Guided Generation for LLMs (Willard & Louf, 2023): the method behind fill-a-fixed-form output.

5. Human approval (pause and ask)

The agent prepares an action but stops and waits for a person to say OK before doing it.

- Flow: User asks, the agent prepares the action, then pauses. The user approves or edits, and only then does the agent carry it out.
- Examples:
  - Email draft: the agent writes it, you approve, then it sends
  - DevOps helper: it suggests a fix and runs it only after you approve
  - Mitra AI: voice order, find items, confirm with the user, place the order, alert family

- Harvey: draft-only cold emails that you check before sending

> READ THE PAPER
> ▸ Human-in-the-loop (LangGraph docs): pause, ask a person, then resume.
> ▸ Building effective agents (Anthropic, 2024): the clearest guide to what an agent is and the common app patterns.

6. Coding / file agent

The agent works on files and runs commands on a computer.

- Flow: You give a task, the agent reads files, runs commands, edits, and checks again until it's done.
- Examples:
  - PR reviewer that reads the changes and writes review comments
  - Docs writer that scans the code and writes the README
  - Bug fixer: run tests, find the cause, edit, re-test
  - Stack: builds the APK, runs emulator tests, and reports what broke and why

> READ THE PAPER
> ▸ SWE-agent: Agent-Computer Interfaces (Yang et al., 2024): an agent that reads code, runs commands, and fixes bugs.
> ▸ SWE-bench (Jimenez et al., 2023): the test used to measure coding agents on real GitHub issues.

7. Team of role agents (crew)

Several agents with job titles work one after another, each passing its work to the next.

- Flow: You give a topic, the researcher gathers facts, the analyst sorts them, and the writer produces the final piece.
- Examples:
  - Blog crew: researcher and writer
  - Market report: researcher, analyst and writer
  - Job apply crew: read the job post, tailor the resume, write the cover letter
  - Stack launch: a marketing crew writes LinkedIn posts, Play Store text and emails

READ THE PAPER

▸ MetaGPT: Meta Programming for Multi-Agent Collaboration (Hong et al., 2023): agents with job titles passing work down a line.

▸ AutoGen: Multi-Agent Conversation (Wu et al., 2023): several agents talking to each other to finish a task.

8. Background / scheduled agent

It runs on its own, on a timer or when an event happens, with no user waiting.

- Flow: The timer fires, the agent runs the loop, saves a report or sends an alert, and the user sees it later.
- Examples:
  - Weekly competitor watch report
  - Daily standup summary from the day's code changes
  - Job leads: reads your leads sheet and drafts one email per company for you to check

READ THE PAPER

▸ Durable execution (LangGraph docs): long jobs that survive restarts and run in the background.

▸ 12-Factor Agents (HumanLayer): rules for building agents that are real, testable software.

9. Voice agent

Same loop, but the user speaks and hears the answer.

- Flow: The user speaks, speech becomes text, the agent loops with its tools, and the answer is spoken back.
- Examples:
  - Voice receptionist that books appointments using a calendar tool
  - Mitra AI voice grocery ordering

READ THE PAPER

▸ Voice agents guide (OpenAI docs): speech in, agent loop, speech out.

▸ Whisper: Robust Speech Recognition (Radford et al., 2022): the speech-to-text step.

10. Mobile app + agent

The phone app has no safe place for secret keys, so it always goes through your server.

- Flow: The user taps a button in the phone app, the request goes to your server, the server checks the user and runs the agent, and the result comes back to the phone. The phone can save it for offline use.

- Example: Stack "Quiz me": the phone asks the server, the agent reads your highlights, and you get a quiz on your phone

> READ THE PAPER
> ▸ OWASP Mobile Top 10: why secret keys must never ship inside a phone app.

11. Local / private agent

The AI model runs on your own computer, so no data leaves it.

- Flow: Same loop, but everything stays on your machine. It usually asks for approval before changing anything.
- Example: a private notes or file assistant running on Ollama

> READ THE PAPER
> ▸ Ollama (GitHub): run a model on your own computer.
> ▸ llama.cpp (GitHub): the engine that makes local models fast.

VI. FIVE RULES EVERY REAL AGENTIC APP FOLLOWS

1. Secret keys stay on the server, never in the website or phone app.
2. The app decides whose data is used. The server passes the logged-in user to the tools, and the agent can never choose another user's data.
3. The agent lives in its own folder, so it's easy to test, improve, or switch models.
4. Answers stream back word by word. Long jobs go to a background worker.
5. Risky actions need human approval: sending, paying, deleting, deploying.

> READ THE PAPER
> ▸ OWASP Top 10 for LLM Applications: the main risks: prompt injection, too much agency, data leaks.
> ▸ Not what you've signed up for: Indirect Prompt Injection (Greshake et al., 2023): why the agent must never pick whose data it reads.

VII. IN ONE LINE

The user asks the app, the server checks who they are and starts the agent, and the agent loops (think, use a tool, read the result) until it has the answer. The answer streams back to the user. Every example above is this same flow with different tools, different numbers of agents, or a pause for approval.

VIII. READING LIST

Start with the first three. The rest go with the section they are linked from.

1. Building effective agents (Anthropic, 2024)
   https://www.anthropic.com/engineering/building-effective-agents
2. LLM Powered Autonomous Agents (Lilian Weng, 2023)
   https://lilianweng.github.io/posts/2023-06-23-agent/
3. A Survey on LLM-based Autonomous Agents (Wang et al., 2023)
   https://arxiv.org/abs/2308.11432
4. ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022)
   https://arxiv.org/abs/2210.03629
5. Toolformer: Language Models Can Teach Themselves to Use Tools (Schick et al., 2023)
   https://arxiv.org/abs/2302.04761
6. MRKL Systems (Karpas et al., 2022)
   https://arxiv.org/abs/2205.00445
7. Tool use with Claude (Anthropic docs)
   https://docs.anthropic.com/en/docs/build-with-claude/tool-use
8. Streaming messages (Anthropic docs)
   https://docs.anthropic.com/en/docs/build-with-claude/streaming
9. 12-Factor Agents (HumanLayer)
   https://github.com/humanlayer/12-factor-agents
10. A Practical Guide to Building Agents (OpenAI, PDF)
    https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf
11. Writing effective tools for agents (Anthropic)
    https://www.anthropic.com/engineering/writing-tools-for-agents
12. Handoffs (OpenAI Agents SDK docs)
    https://openai.github.io/openai-agents-python/handoffs/
13. How we built our multi-agent research system (Anthropic)
    https://www.anthropic.com/engineering/multi-agent-research-system
14. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)
    https://arxiv.org/abs/2005.11401
15. Self-RAG (Asai et al., 2023)
    https://arxiv.org/abs/2310.11511
16. Corrective RAG (Yan et al., 2024)
    https://arxiv.org/abs/2401.15884

17. Introducing Structured Outputs in the API (OpenAI)
https://openai.com/index/introducing-structured-outputs-in-the-api/

18. Efficient Guided Generation for LLMs (Willard & Louf, 2023)
https://arxiv.org/abs/2307.09702

19. Human-in-the-loop (LangGraph docs)
https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/

20. SWE-agent: Agent-Computer Interfaces (Yang et al., 2024)
https://arxiv.org/abs/2405.15793

21. SWE-bench (Jimenez et al., 2023)
https://arxiv.org/abs/2310.06770

22. MetaGPT: Meta Programming for Multi-Agent Collaboration (Hong et al., 2023)
https://arxiv.org/abs/2308.00352

23. AutoGen: Multi-Agent Conversation (Wu et al., 2023)
https://arxiv.org/abs/2308.08155

24. Durable execution (LangGraph docs)
https://langchain-ai.github.io/langgraph/concepts/durable_execution/

25. Voice agents guide (OpenAI docs)
https://platform.openai.com/docs/guides/voice-agents

26. Whisper: Robust Speech Recognition (Radford et al., 2022)
https://arxiv.org/abs/2212.04356

27. OWASP Mobile Top 10
https://owasp.org/www-project-mobile-top-10/

28. Ollama (GitHub)
https://github.com/ollama/ollama

29. llama.cpp (GitHub)
https://github.com/ggml-org/llama.cpp

30. OWASP Top 10 for LLM Applications
https://genai.owasp.org/llm-top-10/

31. Not what you've signed up for: Indirect Prompt Injection (Greshake et al., 2023)
https://arxiv.org/abs/2302.12173

* * *